GRUPO 11

ESPERAS FLUIDAS

Fluent Wait en Selenium

Alexander Molina • Agustín Mites

Taller de QA Moderno (Semana 4)

Java | Selenium WebDriver | Serenity BDD

Presentación técnica | Se complemente con Demo IDE en vivo

SOFKA

¿Por qué fallan los tests?

El problema de los "Flaky Tests" en automatización

Pasa → Falla → Pasa → Falla

⚠️ **Síntomas comunes:**

- NoSuchElementException
- StaleElementReferenceException
- TimeoutException

Desincronización

El test ejecuta comandos más rápido de lo que la UI responde.

DOM Dinámico

Elementos que se re-renderizan o cambian de estado asincrónicamente.

Latencia de Red

Tiempos de respuesta variables en llamadas API o carga de recursos.

El problema no es el código.

Es la **sincronización**. Intentamos interactuar con elementos que aún no están listos.

❌ Anti-patrón común:

```
Thread.sleep(5000); // Espera ciega e ineficiente
```

¿Cómo esperamos de forma inteligente?

Objetivos de la Sesión

Tres pilares para dominar la sincronización en Selenium

Tipos de Espera

Entender las diferencias críticas entre espera Implícita, Explícita y Fluida, y cuándo aplicar cada una según el contexto del proyecto.

FluentWait

Dominar la configuración avanzada: timeouts, polling intervals y manejo de excepciones ignoradas para crear tests robustos.

Buenas Prácticas

Implementar patrones de diseño como Page Object Model (POM) y custom conditions para mantener el código limpio y mantenible.

1 Teoría Fundamentada → 2 Código Real Java → 3 Demo en IDE (Serenity)

Comparativa de Esperas

Implicit vs Explicit vs Fluent: Eligiendo la herramienta correcta

TIPO	ALCANCE	CONFIGURACIÓN	PROS & CONTRAS	CUÁNDO USARLA
Implicit Wait	GLOBAL	Timeout único Polling fijo (500ms)	✓ Simple de configurar ✗ Interfiere con otras esperas	Proyectos muy simples o demos rápidas sin lógica compleja.
Explicit Wait (WebDriverWait)	PUNTUAL	Timeout ExpectedCondition	✓ Precisa por elemento ! Sin control de excepciones	Condiciones estándar estables (visibilidad, clickeabilidad).
Fluent Wait ★ Estrella del Taller	PUNTUAL	Timeout flexible Polling personalizado Ignoring exceptions	Control total granular Verbosidad (código extra)	**UIs dinámicas, AJAX, React/Angular y entornos con latencia variable.**

*Nota técnica: **WebDriverWait** es en realidad una subclase especializada de **FluentWait**. La diferencia clave es que FluentWait expone la API completa para configurar el intervalo de sondeo (polling) y la lista de excepciones a ignorar, permitiendo una robustez superior en situaciones inestables.*

Ciclo de Vida de FluentWait

Entendiendo el algoritmo de espera inteligente

```
INICIO: .until(condición)
        |
        ▼
┌── EVALUAR CONDICIÓN ──┐
|  ¿El elemento existe?  |
|  ¿Es visible/clickable? |
└──────────┬──────────┘
           |
      ┌────┴────┐
      |         |
   CUMPLE     NO CUMPLE
      |         |
      |         ▼
      |   ┌── ¿TIMEOUT AGOTADO? ──┐
      |   |  (Duration.seconds)   |
      |   └──┬──────────┬──────┘
      |      |          |
      |   SÍ |          | NO
      |      ▼          |
      | LANZAR          ▼
      | TIMEOUT   ┌── ¿EXCEPCIÓN IGNORABLE? ──┐
      | EXCEPTION | (NoSuchElement, Stale...) |
      | ✕         └──┬──────────┬──────┘
      |              |          |
```

RITMO

`pollingEvery()` Define la frecuencia del latido. ¿Cada cuánto preguntamos? Evita saturar el navegador con chequeos constantes.

TOLERANCIA

`ignoring()` Lista blanca de errores permitidos durante la espera. Si ocurre uno de estos, no fallamos; simplemente esperamos y reintentamos.

LÍMITE

`withTimeout()` El tiempo máximo total. Si la condición no se cumple en este lapso, el test falla legítimamente con `TimeoutException`.

Parámetros Esenciales

Configurando FluentWait para máxima robustez

```
Wait<WebDriver> wait = new FluentWait<>(driver)
    .withTimeout(Duration.ofSeconds(30)) // ⏱ Límite total
    .pollingEvery(Duration.ofMillis(300)) // 🔄 Frecuencia
    .ignoring(NoSuchElementException.class); // 🛡 Tolerancia

wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("status")));
```

LÍMITE

withTimeout

Tiempo máximo total que el driver esperará. Si se agota sin éxito, lanza `TimeoutException`. Debe ser mayor al tiempo de respuesta p95 del sistema.

RITMO

pollingEvery

Frecuencia de reintento. Define el ritmo del sondeo. Un valor bajo (ej. 100ms) reacciona rápido pero carga la CPU; uno alto (ej. 2s) retrasa el test.

TOLERANCIA

ignoring

Lista de excepciones "permitidas" durante la espera. Es vital para ignorar estados transitorios del DOM (elemento no existe, stale element) sin fallar el test.

ACCIÓN

until

El método disparador. Ejecuta repetidamente la función (Function<WebDriver, T>) hasta que retorne un valor distinto de null/false o expire el timeout.

Arquitectura del Test

Estructura por capas: POM + Serenity + FluentWait

TEST LAYER

QuoteGenerationTest.java (JUnit)

Orquesta el flujo, define aserciones, no conoce selectores.

STEPS LAYER

QuotePageSteps.java (@Step)

Reporte narrativo paso a paso, screenshots automáticos.

PAGE OBJECT MODEL

QuotePage.java

Encapsula UI: Selectores (By.id) + Interacciones.

FluentWait Factory

createWait()

CustomConditions

textDisappears()

WebDriver ↔ Browser

Ventajas Clave

- **Separación de Responsabilidades:** El test dice "qué" probar, el POM sabe "cómo" interactuar.
- **Mantenibilidad Extrema:** Los selectores están en un solo lugar. Si la UI cambia, solo tocas el POM.
- **Lectura de Negocio:** El código del test se lee como una historia de usuario, no como un script técnico.

"Si cambia un ID en la UI
↓
1 cambio en POM, 0 impacto en los tests"

Código Page Object: Factoría FluentWait

Centralización de la lógica de espera y selectores en QuotePage.java

QuotePage.java

```java
public class QuotePage extends PageObject {

// 1. Selectores centralizados (sin hardcodear en tests)
private final By generateBtn = By.id("generate-btn");
private final By quoteStatus = By.id("quote-status");

// 2. Factory Method para FluentWait (DRY - Don't Repeat Yourself)
private Wait<WebDriver> createWait(Duration timeout, Duration polling) {
return new FluentWait<>(getDriver())
.withTimeout(timeout)
.pollingEvery(polling)
.ignoring(NoSuchElementException.class)
.ignoring(StaleElementReferenceException.class);
}

public void clickGenerate() {
getDriver().findElement(generateBtn).click();
}

public void waitForFinalQuote() {
```

Selectores

Definidos como constantes `final By` al inicio de la clase. Si el ID cambia en el HTML, solo se actualiza aquí, no en docenas de tests.

Wait Factory

El método `createWait()` encapsula la configuración compleja. Evita repetir `.ignoring(...)` en cada método de espera, reduciendo el código repetitivo y garantizando consistencia.

Patrón de Diseño: *Centralizar la creación del `Wait` permite cambiar la política de tolerancia global del proyecto en una sola línea (ej. agregar una nueva excepción a ignorar).*

Condiciones Personalizadas: textDisappears

Extendiendo ExpectedCondition para expresar intención de negocio

CustomConditions.java

```java
public class CustomConditions {

// Método estático que retorna una condición reutilizable
public static ExpectedCondition<Boolean> textDisappears(By locator, String text)
{
return driver -> {
try {
WebElement element = driver.findElement(locator);
String currentText = element.getText();
return !currentText.equals(text); // Retorna true si el texto cambió
} catch (NoSuchElementException | StaleElementReferenceException e) {
return true; // Si el elemento desapareció, el texto también
}
};
}
}

// Ejemplo de uso en QuotePage.java:
public void waitForProcessing() {
// Esperamos que "Calculando ..." desaparezca antes de continuar
createWait(Duration.ofSeconds(10), Duration.ofMillis(300))
.until(CustomConditions.textDisappears(By.id("status"), "Calculando ..."));
```

Lógica Negativa

A veces necesitamos esperar a que algo **NO** ocurra. `textDisappears` invierte la lógica: retorna `true` solo cuando el texto ya no coincide.

Manejo de Excepciones

El bloque `catch` es crucial. Si el elemento desaparece del DOM (común en React/Angular al actualizar estados), lanzaría `StaleElementReferenceException`. Aquí atrapamos eso y lo interpretamos como "éxito": si no hay elemento, no hay texto.

Legibilidad: *El test se lee como "esperar hasta que el texto desaparezca", ocultando la complejidad del `try-catch` y el manejo del driver.*

El Test: QuoteGenerationTest.java

Pruebas legibles con intención de negocio (Serenity BDD + JUnit)

QuoteGenerationTest.java

```java
@RunWith(SerenityRunner.class)
public class QuoteGenerationTest {

@Managed(driver = "chrome")
WebDriver driver;

QuotePage quotePage; // Page Object instanciado por Serenity

@Test
public void testQuoteGenerationPassing() {
// 1. Flujo completo en UI correcta
quotePage.openAt("http://localhost:5173/passing");
quotePage.clickGenerate();
quotePage.waitForFinalQuote(); // ⟵ FluentWait oculto aquí
assertThat(quotePage.getQuoteText()).contains("Cotización Total");
}

@Test(expected = TimeoutException.class)
public void testQuoteGenerationIncomplete() {
// 2. Flujo en UI rota (sin elemento status)
quotePage.openAt("http://localhost:5173/incomplete");
quotePage.clickGenerate();
```

Happy Path (Passing)

El test interactúa con la UI completa. El método `waitForFinalQuote()` usa FluentWait para tolerar la latencia y los estados intermedios. Si aparece, el test pasa.

Negative Path (Failing)

Probamos contra una UI rota. Esperamos explícitamente una `TimeoutException`. Esto valida que nuestro FluentWait **no espera eternamente** y falla correctamente cuando debe.

Legibilidad de Negocio: *El test no contiene selectores (`By.id`) ni esperas explícitas. Se lee como un guión de usuario: "Abrir, Clic, Esperar, Verificar".*

Ejecución: Éxito vs Fallo

La robustez no es solo pasar tests, es detectar defectos reales

PASSING

ESCENARIO: UI COMPLETA

El elemento `#quote-status` aparece después de los estados intermedios.

Tiempo real: ~9.2s

```
INFO: Starting testQuoteGenerationPassing
Waiting for #quote-status ...
Polling ... (ignoring NoSuchElement)
Polling ... (ignoring NoSuchElement)
Found element! Condition met.
Tests run: 1, Failures: 0, Errors: 0
BUILD SUCCESSFUL
```

FAILING

ESCENARIO: UI INCOMPLETA

La ruta `/incomplete` nunca renderiza el resultado final.

Timeout agotado: 30.0s

```
INFO: Starting testQuoteGenerationIncomplete
Waiting for #quote-status ...
Polling ... (30s elapsed)
org.openqa.selenium.TimeoutException:
Expected condition failed: waiting for visibility ...
Tests run: 1, Failures: 1
BUILD FAILED
```

Lección clave: *Un test que falla correctamente es tan valioso como uno que pasa. FluentWait no "arregla" la UI rota; expone el defecto real sin confundirlo con problemas de lentitud transitoria.*

Anti-patrones a Evitar

Prácticas comunes que destruyen la estabilidad de los tests

Thread.sleep() como sincronización

Espera "ciega". Si pones 5s y la app carga en 1s, pierdes 4s. Si tarda 6s, el test falla. Lento e inestable.

.ignoring(Exception.class)

Ignorar TODAS las excepciones oculta errores reales (como ElementNotInteractable). Solo ignora lo transitorio.

Mezclar Esperas (Implícita + Explícita)

Causa comportamientos impredecibles en los timeouts del driver. Elige una estrategia y mantenla (preferiblemente Explícita/Fluida).

Timeouts Gigantes (ej. 120s)

"Para que no falle". Resultado: si hay un bug real, esperas 2 minutos por test para enterarte. Frustra el ciclo de feedback.

Selectores Dispersos en Tests

Usar `By.id("btn")` dentro del test en lugar del Page Object. Mantenimiento imposible cuando la UI cambia.

DON'T DO THIS

MAL

```
// ❌ Espera ciega y sucia
Thread.sleep(5000);
// ❌ Tragar errores reales
wait.ignoring(Exception.class);
// ❌ Hardcoding en test
driver.findElement(By.id("btn")).click();
```

"La pereza al escribir el test se paga con horas de debugging."

Buenas Prácticas vs Anti-patrones

La diferencia entre un test frágil y uno robusto

MALOS HÁBITOS

Thread.sleep(5000)
Espera ciega e ineficiente.

ignoring(Exception.class)
Oculta errores reales y bugs.

Timeouts > 120s
Feedback loop demasiado lento.

Selectores Dispersos
Mantenimiento costoso y duplicado.

BUENAS PRÁCTICAS

FluentWait + Condiciones
Espera inteligente hasta que cumpla.

Ignorar solo Transitorias
NoSuchElement, StaleElementRef.

Timeout ajustado (p95)
Falla rápido si el sistema está roto.

Selectores en POM
Centralizados y reutilizables.

REGLA DE ORO

"La espera inteligente no es solo esperar; es saber qué esperar y cuándo dejar de hacerlo."

Reglas de Oro

4 principios fundamentales para tests robustos y mantenibles

1

NO DORMIR EL TEST

Nunca usar `Thread.sleep()`. Es una espera ciega que ralentiza la suite y genera inestabilidad.

Siempre usar condiciones reales sobre el estado del DOM.

2

NO ESCONDER ERRORES

Los errores deben ser visibles. Timeouts gigantes o catchs vacíos ocultan bugs reales en producción.

Un test que siempre pasa (ocultando fallos) no sirve para nada.

3

IGNORAR CON CRITERIO

Solo ignorar excepciones **transitorias** específicas como `NoSuchElement` o `StaleElementReference`.

Nunca ignorar `Exception.class` genérico.

4

CENTRALIZAR LÓGICA

Selectores en **Page Objects**. Waits en **Factory Methods**. Condiciones en **CustomClasses**.

DRY (Don't Repeat Yourself): Si cambia la UI, solo tocas un archivo.

Reflexiones Finales

Conclusiones sobre Fluent Wait y QA Moderno

SOFKA

Cambio de Ritmo

FluentWait transforma la sincronización de una espera pasiva a un sondeo activo e inteligente. No es solo esperar más tiempo, es esperar con ritmo y criterio configurable.

Control Simplificado

No se necesita código complejo. Un simple **Factory Method** de 5 líneas gobierna la política de espera de todo el proyecto, facilitando el mantenimiento y la consistencia.

Aprender a Fallar

Un test robusto debe saber fallar con sentido. FluentWait expone defectos reales (TimeoutException) en lugar de ocultarlos o confundirlos con problemas de red transitorios.

> ***"Un test que nunca falla no sirve para nada. Y un test que falla por timing tampoco. FluentWait nos pone en el punto medio exacto."***
>
> — Grupo 11, Reflexión sobre Automatización Robusta

GRUPO 11

Alexander Molina • Agustín Mites

Taller de QA Moderno (Semana 4)

¿Preguntas?

Gracias por su atención